

Entrevista Técnica — RH PontoCerto (desenhe no draw.io)

Como funciona este exercício. Abaixo está uma **conversa** entre a equipe e o cliente do sistema **RH PontoCerto** (analista de RH de uma empresa média). **Leia com atenção** e, a partir do que as pessoas dizem, **desenhe um diagrama de classes** no draw.io (ou app.diagrams.net). Não é para programar — é para **modelar**.

 **Seu diagrama precisa mostrar os quatro pilares da OO e os três tipos de relacionamento** (associação, agregação e composição). No fim há a **lista do que entregar**, um **lembrete da notação** e um **gabarito** (para o professor).

A entrevista

Gerente: Pessoal, vamos organizar o sistema da **RH PontoCerto**. Vou trazer o cliente pra ele explicar, do jeito dele, o que o sistema precisa ter. Anotem tudo e depois a gente transforma em diagrama.

Cliente: No sistema **RH PontoCerto**, a peça central é **Colaborador**. Tem dois tipos: o **ColaboradorCLT** (o comum) e o **ColaboradorEstagiario**, que **tem carga horária e regras de hora extra diferentes**. No fundo, os dois são um tipo de **Colaborador** — todos têm nome e matrícula e compartilham a mesma base.

Dev sênior: Entendi. Então tem uma parte **igual pra todos** e uma parte **específica** de cada tipo. Guardem isso — cheira a uma base comum com dois tipos que herdam dela.

Cliente: Isso. E o comportamento muda por tipo: olhando o **calcularAdicional()** de cada um, pro **ColaboradorCLT** hora extra pela regra CLT; já pro **ColaboradorEstagiario** regra de estágio (carga reduzida). A **mesma operação, resposta diferente conforme o tipo**.

Analista de qualidade: Um ponto sério: as horas não podem ser mexidas por fora (marcação inválida, como 30:00, é recusada). Ninguém pode mexer nesses dados **por fora** — tem que ser por **operações controladas**. E tem uma regra que **NÃO pode falhar: não pode registrar saída antes da entrada no mesmo dia**. Por isso o estado de **Ponto** só anda por operações (aberto (entrada) → fechado (saída); ou ajustado), nunca "na mão".

Cliente: Agora as ligações. Um colaborador tem MUITOS registros de ponto e pertence a UM departamento.

Cliente: Tem uma ligação de **parte-todo que nasce e morre junto: Colaborador ♦ RegistroPonto**. Cada marcação pertence ao colaborador e some junto se o cadastro for removido. (pense em **composição** — losango cheio ♦)

Dev sênior: Não confunda com o outro caso: **Departamento** ◇ **Colaborador** é só um **agrupamento**. Os colaboradores existem por conta própria; o departamento só os agrupa. Se **Departamento** sumir, **Colaborador** continua existindo — isso é **agregação** (losango vazio ◇), bem diferente da composição.

Gerente: Fechou. Com isso dá pra desenhar o modelo. Mãos à obra.



Dicas de leitura (pistas escondidas na conversa)

O que apareceu na conversa	Onde isso te leva
"os dois são um tipo de Colaborador (mesma base)"	um tipo geral (base) e tipos específicos → herança
"a mesma operação calcularAdicional() , resposta muda por tipo"	operação redefinida em cada tipo → polimorfismo
"As horas não podem ser mexidas por fora (marcação inválida, como 30:00, é recusada)"	dado privado + operações → encapsulamento
"não pode registrar saída antes da entrada no mesmo dia"	invariante : o estado muda só por operação que valida
" Colaborador ◆ RegistroPonto (nascem e somem juntos)"	composição (losango cheio ◆)
" Departamento ◇ Colaborador (Colaborador existe sozinho)"	agregação (losango vazio ◇)
"um colaborador tem MUITOS registros de ponto e pertence a UM departamento"	associação com multiplicidade



O que você deve entregar (no draw.io)

Um **diagrama de classes** que contenha, de forma clara:

1. **Abstração** — uma classe **base** que não é criada diretamente (ex.: **Colaborador**).
2. **Herança** — **ColaboradorCLT** e **ColaboradorEstagiario** ligados à base pelo triângulo vazio →.
3. **Encapsulamento** — atributos sensíveis como **privados** (−) e **operações públicas** (+) que os controlam (proteger o valor e o estado da **Ponto**).
4. **Polimorfismo** — a operação **calcularAdicional()** declarada na base e **redefinida** em cada tipo.
5. **Os três relacionamentos**, cada um com sua **multiplicidade**:
 - **Associação** (linha/seta): um colaborador tem MUITOS registros de ponto e pertence a UM departamento.
 - **Agregação** (losango **vazio** ◇): **Departamento** junta **Colaboradors** que existem sozinhos.
 - **Composição** (losango **cheio** ◆): **Colaborador** é feita de **RegistroPontos** que nascem e morrem com ela.

Lembrete de notação (UML)

Elemento	Como desenhar
Classe abstrata	nome em <i>itálico</i> + « abstract » (não se cria direto)
Operação abstrata	em <i>itálico</i> (cada subtipo redefine)
Visibilidade	– privado · + público · # protegido
Herança	linha com triângulo vazio  apontando para a base
Associação	linha (com seta), com multiplicidade nas pontas
Agregação	losango vazio  no lado do "todo"
Composição	losango cheio  no lado do "todo"

Multiplicidades: **1** (exatamente um), **0..1** (zero ou um), ***** (muitos), **1..*** (um ou mais).

Critério de "pronto"

- ☐ Há uma **classe base abstrata** e os dois tipos herdando dela.
- ☐ Nenhum atributo sensível está público; o valor e o estado da **Ponto** são protegidos.
- ☐ A operação **calcularAdicional()** aparece na base e é **redefinida** em cada tipo.
- ☐ Aparecem os **três** relacionamentos: **associação**, **agregação** () e **composição** ()
- ☐ Você consegue **explicar** por que **Colaborador**→**RegistroPonto** é composição e **Departamento**→**Colaborador** é agregação.
- ☐ Todas as ligações têm **multiplicidade**.

 [Briefing do projeto](#) ·  [Plano de evolução](#)